



MANIFESTO / TECHNICAL PRIMER

A complete technical and philosophical account of what Parity is, how it works, and what it deliberately does not claim.

CONTENTS

01	Why Parity Exists
02	The Parity Model
03	Data Sources
04	Snapshot & History Architecture
05	Data Health vs. Statistical Eligibility
06	Historical Baseline
07	Maturity
08	Classification
09	Persistence & Episodes
10	Alert Engine
11	Risk API
12	Execution / Liquidity Telemetry
13	Gas Telemetry
14	Product Principles
15	Limitations
16	Future Direction

Robinhood Chain Stock Tokens create on-chain markets for assets whose value is meant to track something else entirely — a reference price, observed elsewhere, updated on its own schedule. The two prices are not required to move together at every moment. An on-chain price can drift above its reference (a premium) or below it (a discount), and nothing about the architecture guarantees the gap stays small.

Seeing that a gap exists is trivial: read two numbers, subtract. Understanding what the gap *means* is the actual problem. A premium that appears and disappears within an hour is not the same condition as a discount that persists for a day. A deviation that is unremarkable for one Stock Token may be extreme for another, because different tokens have different baseline behavior. A single percentage, on its own, carries no history and no context — and history is exactly what turns "the price is different" into something useful.

Parity exists to answer one question, continuously, for every Stock Token it supports:

"Is this Stock Token still trading at parity with what it is supposed to represent?"

Not "is the price different" — that's the easy half. The harder half is distinguishing ordinary basis (small, temporary, unremarkable divergence) from genuine dislocation (a reading that is statistically abnormal relative to that specific token's own recent behavior). Parity's entire design exists in service of that distinction, and every section below traces back to it.

02 THE PARITY MODEL

Parity is the point of exact correspondence between a Stock Token's on-chain price and its reference price — 0.00% deviation. **Premium** means the on-chain price sits above the reference; **discount** means it sits below. **Deviation** is the signed percentage distance between the two.

Direction is descriptive, not evaluative. A premium is not "good" and a discount is not "bad" — they describe which side of the line a price currently sits on, nothing more. This distinction matters because it is easy, especially in a market context, to unconsciously read "premium" as bullish and "discount" as bearish. Parity does not support that reading, and nothing in its output should be interpreted that way.

The **Parity Line** is the visual and conceptual anchor for this whole model — the 0.00% axis, with discount on one side and premium on the other:



Everything Parity reports is ultimately a statement about distance from that line, and — critically — about what that distance has looked like over time for this specific Stock Token. A raw deviation reading answers "how far from the line, right now." Everything from Section 06 onward exists to answer the much more useful question: "is that distance ordinary or abnormal, for this token, based on its own history?"

Parity's intelligence is built entirely from data that is independently observable and verifiable — no data source is invented, estimated, or purchased from a proprietary feed. Confirmed current sources:

Robinhood registry and price data. Robinhood's own published `/rhj/assets` (registry, asset status, multiplier state) and `/rhj/prices` (bid/ask quote, trading-halt flag) endpoints. This is the source for the Stock Token's own live quote and operational status.

Chainlink reference feed. An on-chain `AggregatorV3`-style price feed, read directly from Robinhood Chain, per supported ticker. This is what Parity compares the on-chain price against. It is deliberately never described as "the real price" — it is a reference, chosen for being independently verifiable on-chain, not a claim of ground truth about the underlying equity's actual market value.

`oraclePaused()`. A boolean read directly from the Stock Token contract itself, indicating whether its own price oracle is currently paused. When true, Parity treats the token's pricing as untrustworthy for that observation, regardless of what any other source reports.

Uniswap V3 pricing. The on-chain price actually being evaluated. Pool addresses are resolved authoritatively via the Uniswap V3 factory contract (`factory.getPool(tokenA, tokenB, feeTier)`) — never computed offline and never trusted purely from configuration without that live resolution step. The pool's `slot0()` call supplies the current `sqrtPriceX96` and `tick` in one read.

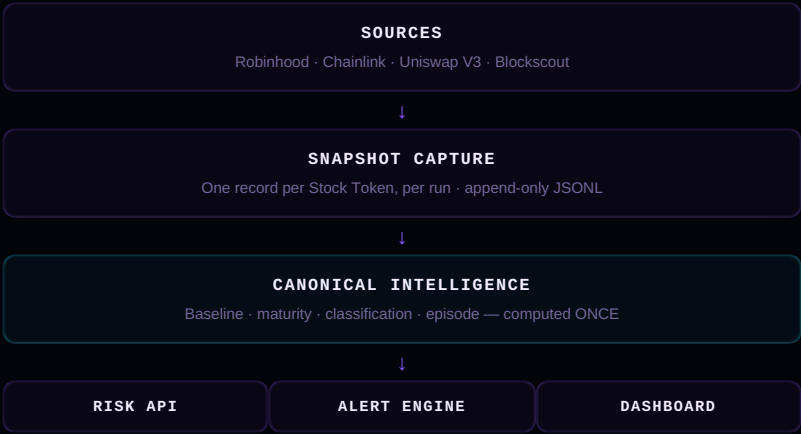
Blockscout holder data. Holder-concentration statistics for context on token distribution. This source degrades gracefully — if the required API key isn't configured, holder data is reported as unavailable rather than omitted silently or blocking anything else.

Gas telemetry. A periodic, ticker-independent sample of chain gas conditions (Section 13).

Each source is read independently, and a failure in one never silently corrupts or fabricates another — this per-field independence is a structural property of the snapshot itself (Section 04), not just a design intention.

04 SNAPSHOT & HISTORY ARCHITECTURE

Parity's collector runs on an intended 15-minute cadence, capturing one snapshot record per supported Stock Token per run. Every record is appended to a durable, append-only JSONL file — never overwritten, never deleted, never rewritten in place. This gives Parity a genuine historical dataset that accumulates value over time: the longer Parity runs, the more precisely it can characterize what "normal" looks like for a given Stock Token, which is the entire foundation the statistical model in Sections 06–08 depends on.



A malformed or truncated line in the historical file does not corrupt the rest of the dataset — the reader skips unparseable lines individually rather than failing the whole read. This matters operationally: a partial write during a crash cannot silently destroy months of accumulated history.

Parity never silently interpolates a missing historical value. If a collection run fails for a given ticker, that observation is simply absent from the record — not filled in with an estimate, not carried forward from the last known value, not smoothed over. This is a deliberate constraint with real consequences: it means Parity's persistence-episode logic (Section 09) must explicitly handle gaps rather than assume continuity, and it means a reader can trust that every data point in the historical record reflects an actual observation, never a fabricated one.

05 DATA HEALTH VS. STATISTICAL ELIGIBILITY

This is one of the most important — and most easily conflated — distinctions in the entire system. Parity maintains **two separate, layered checks**, and collapsing them into a single concept would misrepresent how the product actually works.

Layer 1 — raw data health. Every snapshot is classified into one of five states: `healthy_current`, `stale_reference`, `oracle_paused`, `trading_halt`, or `upstream_unavailable`. This layer answers a narrow question: is the underlying data itself currently trustworthy and fresh enough to display at all?

Layer 2 — statistical eligibility. A stricter, separate gate layered on top of Layer 1. Even a Layer-1-healthy reading must additionally satisfy a tighter freshness requirement — the reference must be no older than **360 minutes** — before it is trusted as an input to the statistical baseline and classification machinery. A reading can be perfectly healthy by Layer 1's standard while still being ineligible by Layer 2's stricter one.

The practical consequence: **a current deviation can be visible on the dashboard or API while classification is simultaneously unavailable**. This is not a bug or an inconsistency — it is the two layers doing exactly what they are designed to do. The deviation only requires Layer-1 health; classification additionally requires Layer-2 eligibility *and* a mature baseline (Section 07). Any explanation of Parity that describes eligibility as a single yes/no concept is incomplete.

06 HISTORICAL BASELINE

From a Stock Token's accumulated, statistically-eligible observations, Parity builds a **baseline**: a historical **median** deviation, and a **baseline dispersion** describing how much that deviation typically varies around its median. The dispersion statistic is the **MAD — median absolute deviation**.

Median and MAD were chosen deliberately over mean and standard deviation because they are resistant to distortion by a small number of extreme readings. A handful of unusual observations in a token's history should not silently drag the entire baseline off-center the way a few outliers can distort a mean — median and MAD hold steady in exactly that scenario, which matters because the whole point of the baseline is to represent *typical* behavior.

A small technical safeguard exists here: **MAD_FLOOR_PCT = 0.005**. If a token's real historical dispersion happens to be extremely small (or briefly zero), dividing by that tiny number would produce an artificially enormous multiple for even a modest deviation — a floor prevents that distortion without changing what the statistic means.

Formula, for readers who want it (not required to understand the system):

```
MAD multiple = |current deviation - historical median| / max(baseline dispersion, MAD_FLOOR_PCT)
```

The result — the **MAD multiple** — measures how far the current deviation sits from the historical median, expressed as a multiple of the baseline dispersion. It is the single number classification (Section 08) is actually measured against. It is never referred to as a z-score (it is a robust, median-based statistic, not a mean/standard-deviation one), and never described using informal "x typical" phrasing.

07 MATURITY

A baseline is only as trustworthy as the history behind it. Parity tracks **maturity** to ensure classification is never produced from a baseline too young or too thin to mean anything — a form of protection against false precision.

INSUFFICIENT_DATA — fewer than 20 observations, or less than 24 elapsed hours of history. No baseline exists yet; nothing is computed.

DEVELOPING — a baseline exists but hasn't yet cleared the MATURE bar below. Deviation may still be genuinely visible at this stage (Section 05 explains why), but classification, MAD multiple, and episode detail are not yet produced from it.

MATURE — at least **200 observations AND** at least **96 elapsed hours**. Both conditions are required together, not either alone. Only a MATURE baseline produces a classification.



Only a MATURE baseline produces a classification.

This gating exists because a thin history cannot yet reliably represent "typical" behavior for a token — reporting a confident-sounding classification from a handful of observations would manufacture a false sense of precision the underlying data doesn't actually support. A young Stock Token is honestly reported as still developing, not forced into a premature verdict.

08 CLASSIFICATION

Once a baseline is MATURE and the current observation is eligible, Parity classifies the current deviation into one of four tiers, based on its MAD multiple **and** its absolute magnitude — both conditions required together, not either alone:

TIER	MAD MULTIPLE	ABSOLUTE DEVIATION
NORMAL	below all thresholds below	—
ELEVATED	$\geq 2.0\times$	$\geq 0.10\%$
DISLOCATED	$\geq 3.5\times$	$\geq 0.25\%$
SEVERE	$\geq 5.0\times$	$\geq 0.50\%$



The dual condition exists because relative abnormality alone is not sufficient information. A token with an extremely tight historical baseline could register a huge MAD multiple from a deviation that is, in absolute terms, negligible — the absolute floor prevents classification from firing on statistically-large-but-practically-meaningless movement. Conversely, the relative condition prevents a token with naturally wide historical dispersion from being classified abnormal simply because its ordinary variation happens to be numerically larger than another token's.

CRITICAL CAVEAT – read this before quoting any threshold above:

These four tiers and their exact thresholds are current operating hypotheses about relative abnormality — **not empirically validated risk boundaries**. They describe distance from a Stock Token's own history, not a proven measure of danger, and they have not yet been validated against long-run outcome data. Treat classification as a relative-abnormality signal, never as a validated risk score.

A single abnormal reading and a sustained abnormal condition are genuinely different situations, and collapsing that distinction would discard one of the more useful parts of the signal. Parity tracks this via **episodes** — a continuous run of non-NORMAL classification for a given Stock Token.

An episode's continuity is governed by explicit, honest rules, not assumption:

- **The observation-gap rule.** If the time between two consecutive real records exceeds **30 minutes**, they are not treated as temporally continuous — a genuine collection gap correctly breaks episode continuity rather than being silently bridged. Parity never assumes an abnormal condition persisted through a gap it has no actual observation for.
- **Invalid or ineligible readings break continuity.** A reading that fails eligibility does not count as "still abnormal" — it is treated honestly as a break, not folded into the surrounding episode.
- **A NORMAL reading always ends an episode.** By definition, an episode is a run of non-NORMAL states; the moment a NORMAL reading occurs, the episode ends.
- **No interpolation, ever.** A gap in the data is a gap — never smoothed, never filled with an assumed value, never treated as if it didn't happen.

Duration matters because it changes what a reading actually tells a reader. A dislocation that has held for six hours is a materially different situation from one that appeared once and vanished, even if the instantaneous classification looks identical in both cases. Episodes are how Parity preserves that difference rather than flattening it into a single point-in-time label.

10 ALERT ENGINE

Parity's alert engine is a pure consumer of already-computed canonical intelligence — it does not compute a second, independent model of risk. It reads the same `RiskViewModel` the API and dashboard read, compares the current classification to the immediately preceding one for that Stock Token, and decides whether a specific kind of transition occurred.

Three transitions produce an alert:

- `NORMAL` → `abnormal` → `new_risk`
- `abnormal` → `more severe abnormal` → `escalation` (e.g. `ELEVATED` → `DISLOCATED`)
- `abnormal` → `NORMAL` → `recovery`

NORMAL	→	ABNORMAL	new_risk
ABNORMAL	→	MORE SEVERE	escalation
ABNORMAL	→	NORMAL	recovery
ABNORMAL	→	LESS SEVERE	no alert
null	→	anything	no alert

No alert fires for: the same classification repeated; a downgrade between two abnormal tiers (e.g. `SEVERE` easing to `DISLOCATED`) — a deliberate choice, since inventing a "partial improvement" alert without a designed, validated semantic for it would be worse than staying silent; or any transition where either the previous or current classification is null, since a gap in visibility cannot be reliably distinguished from a genuine transition.

Deduplication is durable, not in-memory. Parity records the timestamp of the last snapshot it evaluated for each Stock Token, persisted to disk. A snapshot at or before that recorded timestamp is treated as already-processed and never re-evaluated — this behavior survives a process restart, since the check is against durably-stored state, not memory that would reset on restart.

Delivery defaults to a structured log line; an optional Discord webhook can be configured as an alternative delivery target. **This is explicitly not a reliable-delivery system today:** if a Discord delivery attempt fails after alert state has already durably advanced (which happens before delivery is even attempted), the event cannot currently be recovered by replaying the same snapshot — deduplication will correctly, if unhelpfully in this specific case, treat that snapshot as already handled. A durable outbox/retry mechanism is a deliberately deferred, later concern, not a claim this document should make about current behavior.

11 RISK API

`GET /api/v1/risk/:symbol` exposes exactly the same canonical intelligence used internally — it does not run a separate computation path. The response includes the current deviation and its direction, a P2 reference-status field, a P3 eligibility boolean, maturity, the baseline statistics (historical median, baseline dispersion, MAD multiple), classification, and active episode detail where available.

`apiVersion: "v1"` appears on every response, success or error. The rate limit is **60 requests per minute per client IP**, enforced only on this route; exceeding it returns HTTP 429 with a truthful `Retry-After` header. An unsupported symbol returns `unsupported_ticker` (404); a supported symbol with no captured data yet returns the deliberately distinct `no_data_yet` (404) — "not available yet," never "will never exist here."

The eligibility field is intentionally a plain boolean today, without a granular reason, because the underlying engine does not yet preserve the specific distinction between a Layer-1 health failure and a Layer-2 eligibility failure all the way through to this contract. Inventing a plausible-sounding reason the product cannot actually stand behind would be worse than the current honest boolean.

12 EXECUTION / LIQUIDITY TELEMETRY

Parity has begun preserving raw execution/liquidity telemetry per supported Stock Token, per snapshot:

`poolSqrtPriceX96` and `poolTick` (preserved from the same pool read the pricing logic already performs, at zero additional cost), `poolFeeTierBps` (a known configuration constant), `poolLiquidity` (the pool's active liquidity at its current tick), and `poolToken0Balance` / `poolToken1Balance` (the pool contract's raw reserve balances).

RAW TELEMETRY ≠ EXECUTION INTELLIGENCE

A Uniswap V3 pool's liquidity is concentrated at specific price ticks, not spread evenly across all prices — `poolLiquidity` describes what's active near the current tick, not the pool's whole story. Raw token balances are the pool's *total* reserve across every tick, which can substantially overstate what is actually tradeable near the current price. None of these raw fields have been converted into a slippage estimate, a liquidity score, or an execution score anywhere in the product today, and none should be read as one.

This telemetry is being collected now specifically because it cannot be reconstructed retroactively — any historical pool state before this capability existed is permanently unavailable. Collecting it preserves the option to build genuine execution/liquidity intelligence later, without pretending that capability already exists today.

13 GAS TELEMETRY

Independently of any specific Stock Token, Parity samples general chain conditions once per collection run: block number, block timestamp, and base fee per gas. This is general network-fee context — it does not measure per-transaction cost, per-ticker gas usage, or anything Stock-Token-specific, and it should not be read as measuring more than what it actually samples.

Independent. Parity is not affiliated with Robinhood. It observes public, verifiable data and reports what it finds.

Read-only. Parity does not execute trades, custody assets, or take any action on a user's behalf.

Data-honest. Unavailable data is reported as unavailable — never silently substituted with zero, never quietly filled in with a guess. An exact 0.00% reading is a real, distinct measurement, never confused with "no data."

No financial advice, no buy/sell signals, no guaranteed arbitrage. Every classification, deviation, and alert describes an observed condition relative to history — never an instruction, a prediction, or a recommendation.

No invented certainty. Where the product genuinely doesn't know something — a granular eligibility reason, a slippage figure, a validated risk boundary — it says so rather than manufacturing a plausible-sounding answer it cannot stand behind.

One canonical intelligence model, reused everywhere. The dashboard, the Risk API, and the alert engine all read the same underlying computation. Nothing is independently recomputed for a different surface, which means there is exactly one place classification logic can be wrong, and exactly one place it needs to be right.

15 LIMITATIONS

Stated plainly, without hedging:

- Only **six** Stock Tokens are currently supported (AAPL, GOOGL, USO, SPCX, TSLA, NVDA) — this reflects which pools are currently independently verified, not a ceiling on the architecture.
- Pricing is **Uniswap V3 only**; there is no V4 support today.
- Classification thresholds are unvalidated operating hypotheses, not proven risk boundaries.
- The Risk API's **eligibility** field does not currently expose a granular reason.
- No slippage engine, liquidity score, or execution score exists in the product today.
- Alert delivery has no retry or durable outbox mechanism — a failed delivery after state has advanced is not currently recoverable.
- Historical execution/liquidity telemetry before P5-C0 was never captured and cannot be reconstructed.
- Upstream data freshness and market-hour behavior depend entirely on the underlying sources (Chainlink, Robinhood, the pool itself) — Parity's own collector runs continuously, but a source updating (or not) outside traditional market hours is a property of that source, not something Parity controls.

16 FUTURE DIRECTION

Everything in this section is planned or aspirational — none of it is current capability, and nothing here should be read as an existing feature.

Broader Stock Token coverage, as more pools are independently verified. Richer market-integrity intelligence beyond the current four-tier classification. Genuine execution/liquidity intelligence built on top of the raw telemetry already being preserved (Section 12) — but only once that capability is deliberately designed and validated, not assumed from the existence of raw data. Deeper historical analytics as the accumulated dataset grows. Institutional and API-driven use cases as the developer contract matures. Expanded alert semantics, potentially including the currently-silent downgrade case (Section 10), once a validated design exists for it.

None of this changes what Parity is today: a read-only instrument that observes verified data and reports statistically-grounded context about it — nothing more, and nothing invented ahead of what the data can actually support.

PARITY · READ-ONLY INTELLIGENCE OVER VERIFIED ON-CHAIN DATA